# CANopen Measurement Gateway
# - Project Introduction -

## 1 Preface

The Measurement Gateway development requires Linux as operating system as we need several network protocol stacks, symmetric multiprocessing and several processes/threads for the application.

The development tool (Vivado, Petalinux and Vitis) are bound to the installed operating system version. Therefore, do not update the Linux as this will break the dependencies and makes all these tools unusable.

*You must not install any additional software.* The installed packages and libraries are proved to be complete for the development process.

However, application libraries like the CANopen stack can be downloaded and used for your application development on the target system (ZedBoard).

**Node: paelx1**

**User: dmplab (ESD PAElab1)**

**Path for development: ~/ProjCanO** (default path on login)

Do not use any other folders for your work. Create only subfolders within ~/ProjCanO.

Folder outline:

```
ProjCanO/
├── DOCs          documents (including this)
└── mcando        top level design folder
    ├── Archive   holds files worth to keep
    ├── hsmcand   HW/SW design (Vivado / Vitis)
    │             => This is the place for your development
    └── lxmcand   Linux Kernel, rootfs, boot files (hughe folder!)
                  => Needed for adding applications to the rootfs
                     (not for developing and test of software applications)
```

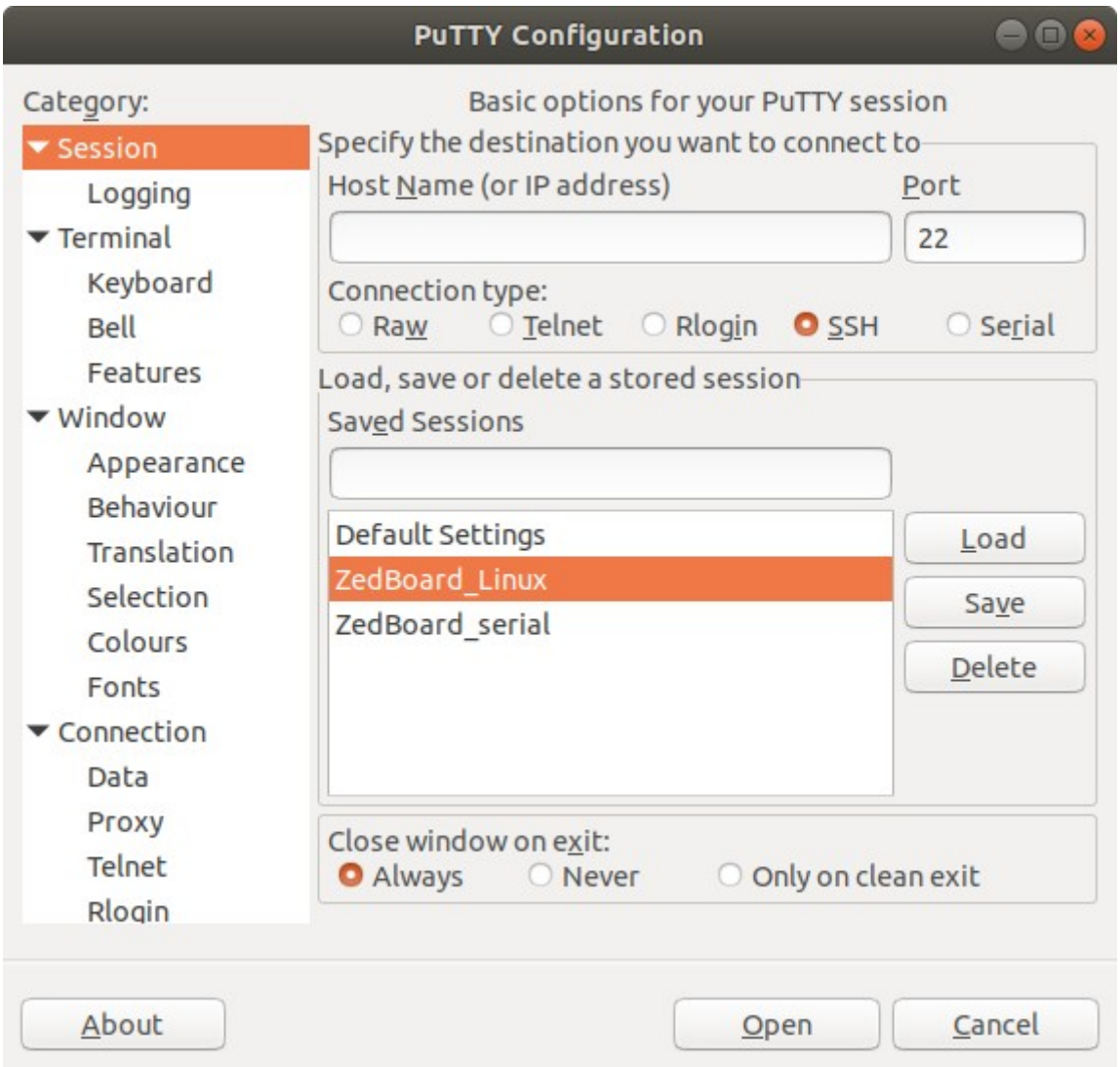
Backup of all design files can be done with the command:

**proj_backup**

This will create a compressed tar image on a separate hard disk. Restoring of files have to be done manually when needed.

## 2 Terminal Connection to the Target System

Use **putty** to get a terminal to the ZedBoard. Putty allows SSH and serial connections.




Double-click on **ZedBoard_Linux** for a Linux terminal or double-click **ZedBoard_serial** for bare metal application tests. The board must be powered on to do this. Linux will automatically boot from SD-card. If you want to disable booting Linux, remove the SD-card.

If the putty terminal window does not open, repeat the above steps until it opens. Reason for this behavior is the USB connection which requires some time to detect the device.

**Bare metal applications:** no action required.

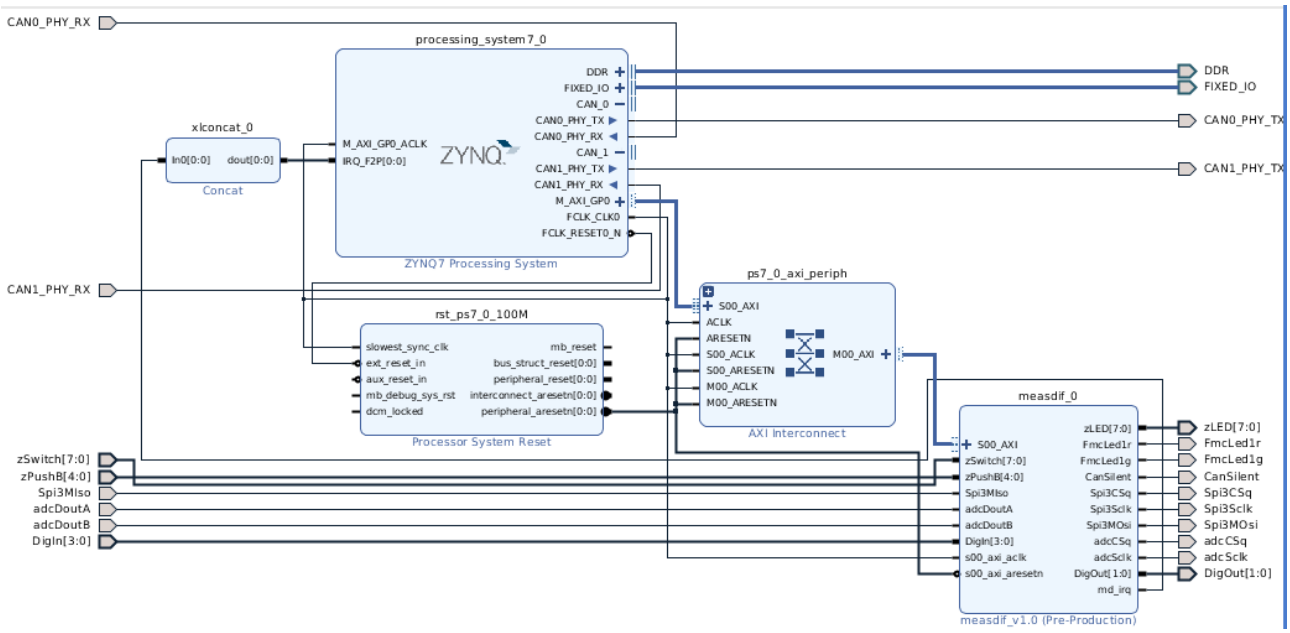
**Linux:** Wait for the booting process to complete. If it stops in the u-boot boot loader, enter **boot** to continue the booting process. You can login then a user: **root** with password **root**.

```
/dev/ttyACM0 - PuTTY
PetaLinux 2020.1 lxmcand /dev/ttyPS0

lxmcand login:
PetaLinux 2020.1 lxmcand /dev/ttyPS0

lxmcand login:
PetaLinux 2020.1 lxmcand /dev/ttyPS0

lxmcand login:
PetaLinux 2020.1 lxmcand /dev/ttyPS0

lxmcand login:
PetaLinux 2020.1 lxmcand /dev/ttyPS0

lxmcand login: root
Password:
Last login: Mon Nov  2 15:57:43 UTC 2020 on ttyPS0
root@lxmcand:~# ls -l /usr/bin/v*
-rwxr-xr-x    1 root     root          5572 Nov  2 15:56 /usr/bin/vitiststt
lrwxrwxrwx    1 root     root            17 Nov  2 15:57 /usr/bin/vlock -> /bin/
busybox.suid
root@lxmcand:~# vitiststt
--- Vitis Linux Development Test passed. ---
root@lxmcand:~#
```

# 3 System Setup

## 3.1 Hardware Design

Hardware design is a matter of Vivado (hardware only). The figure shows the block design. It consists of the ZYNQ ARM A9 Core and the custom IP “measdif”. This IP contains the logic for the measurement board and includes a timer for fabric interrupt (IRQ 61).



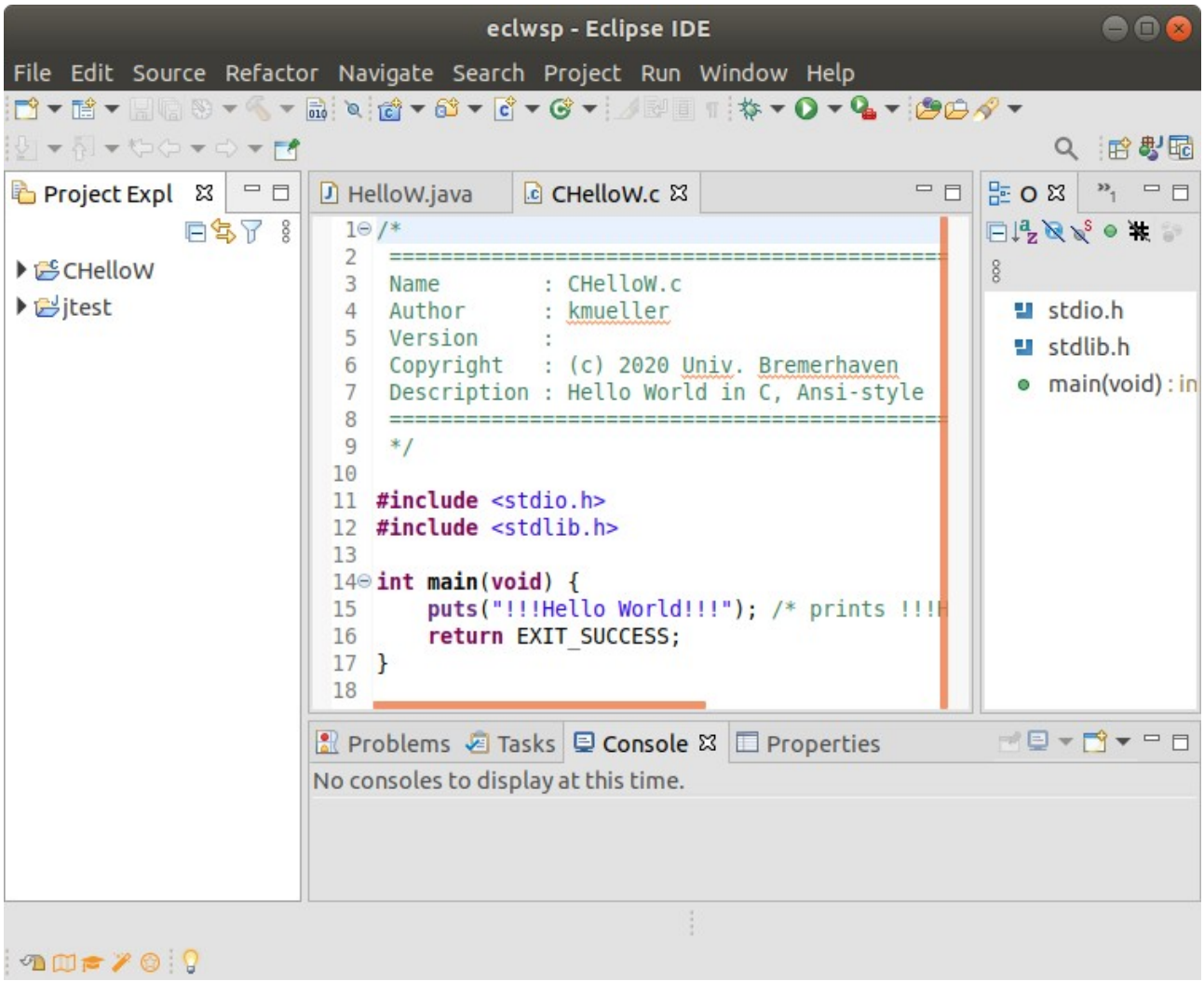

The hardware (including bitstream) is exported (for SW design).

*Whenever hardware is changed the export has to be done again.*

## 3.2 Software Design

### 3.2.1 Vitis

It is a good approach to test parts of your C code in the Desktop  Eclipse environment first.

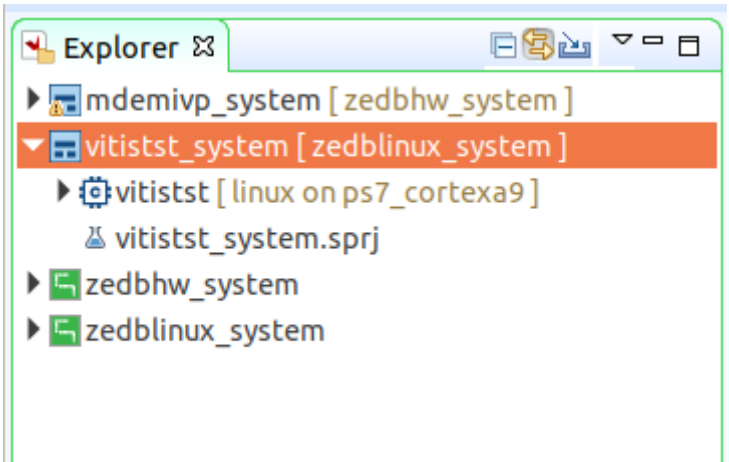



Since Embedded Linux and Desktop Linux are almost identical with respect to SW development, some modules can be easier tested on Desktop Linux.

---

**Software design is a matter of Vitis** (Vitis allows the use of several platforms and several applications on these platforms).

Two platforms are currently created in Vitis:



**zedbhw_system**: for bare metal applications

**zedblinux_system**: for linux applications

When creating new “C” applications you have to select one of these platforms.

Here is an **example for a bare metal program:**

---

```c
/*
** mdivpmn.c: Process IO Partial Tests
*/

#include <stdio.h>
#include <string.h>
#include "platform.h"
#include "xil_printf.h"
#include "xparameters.h"
#include "xscugic.h"
#include "xil_exception.h"


#define MDHW_REG(k) *(volatile unsigned int *)
(XPAR_MEASDIF_0_S00_AXI_BASEADDR+4*k)
#define N_REGS 4

#define TIMER100ms  0x009896ff
#define MBINT_ENABLE 0x80000000
#define MBINT_ACKN 0x40000000

static unsigned int R0_Copy, R1_Copy, R2_Copy, R3_Copy;

// ---- interrupt controller -----
static XScuGic  Intc;                          // interrupt controller instance
static XScuGic_Config  *IntcConfig;       // configuration instance

static volatile unsigned int Led_Output;
static volatile unsigned int ISR_Count;
static volatile unsigned int Do_Display;

#define CBUF_LEN 64


/*
** ------------------------------------------------------------
** Interrupt handler (iqr61 fabric interrupt)
** ------------------------------------------------------------
*/
static void FabricIntrHandler(void *CallBackRef)
{
      // int  ih_par = *(int *)CallBackRef;
      R2_Copy |= MBINT_ACKN;
      MDHW_REG(2) = R2_Copy;
      MDHW_REG(0) = Led_Output;
      Led_Output <<= 1;
```

```
      if ((Led_Output & 0x0FF) == 0) {
            Led_Output = 1;
      }
      Do_Display = 1;
      ISR_Count++;
      R2_Copy &= ~MBINT_ACKN;       // rearm interrupt irq61
      MDHW_REG(2) = R2_Copy;
}


/*
 * Receive data from ADC (2 x 16 Bit)
 */
static void ADCRead()
{
      int  k;
      unsigned int retc, adcdat, adcA, adcB;
      int  ai_0, ai_1;

      k = 0;
      MDHW_REG(3) = 0;
      do {
            retc = MDHW_REG(2);
            k++;
      } while (((retc & 0x01) == 0) && (k < 500));
      adcdat = MDHW_REG(3);
      adcA = adcdat & 0x0FFFF;  adcB = adcdat >> 16;
      printf(" o ADC_B: %x  ADC_A: %x  (%8x) [%d retries]\n\r", adcA, adcB,
adcdat, k);
      ai_0 = adcA & 0x0FFF;
      if ((adcA & 0x0800) != 0) {
            ai_0 |= 0xFFFFF000;
      }
      ai_0 = -ai_0;
      ai_1 = adcB & 0x0FFF;
      if ((adcB & 0x0800) != 0) {
            ai_1 |= 0xFFFFF000;
      }
      ai_1 = -ai_1;
      printf(" o Ai_0: %d  Ai_1: %d\n\r", ai_0, ai_1);
}



int main()
{
      int  int_par, terminate, isr_run, regn, repeatc, k;
      unsigned int xdata;
      char cbuf[CBUF_LEN], *chp;

    init_platform();
    printf("--- MeasDem Tests V0.2a---\n\r");
    R0_Copy = 0;
    R1_Copy = 0;
      R2_Copy = TIMER100ms;
      R3_Copy = 0;

    terminate = 0;
    ISR_Count = 0;
      Do_Display = 0;
    isr_run = 0;
      Led_Output = 1;
```

```
    printf(" * initialize exceptions...\n\r");
    int_par = XPAR_FABRIC_MEASDIF_0_MD_IRQ_INTR;
    Xil_ExceptionInit();

    printf(" * lookup config GIC...\n\r");
    IntcConfig = XScuGic_LookupConfig(XPAR_SCUGIC_0_DEVICE_ID);
    printf(" * initialize GIC...\n\r");
    XScuGic_CfgInitialize(&Intc, IntcConfig, IntcConfig->CpuBaseAddress);

      // Connect the interrupt controller interrupt handler to the hardware
    printf(" * connect interrupt controller handler...\n\r");
      Xil_ExceptionRegisterHandler(XIL_EXCEPTION_ID_IRQ_INT,
                          (Xil_ExceptionHandler)XScuGic_InterruptHandler, &Intc);

    printf(" * set up fabric interrupt...\n\r");
    XScuGic_Connect(&Intc, XPAR_FABRIC_MEASDIF_0_MD_IRQ_INTR,
(Xil_ExceptionHandler)FabricIntrHandler,
                          (void *)&int_par);
    // level triggered interrupt
    XScuGic_SetPriorityTriggerType(&Intc, XPAR_FABRIC_MEASDIF_0_MD_IRQ_INTR,
0x00, 0x01);
    print(" * enable interrupt for IRQ61 at GIC...\n\r");
    XScuGic_Enable(&Intc, XPAR_FABRIC_MEASDIF_0_MD_IRQ_INTR);

    MDHW_REG(2) = R2_Copy;          // load interrupt timer

      // Enable interrupts in the Processor.
    printf(" * enable processor interrupts...\n\r");
      Xil_ExceptionEnable();

      do {
      print(">> "); fflush(stdout);
      fgets(cbuf, CBUF_LEN, stdin);
      cbuf[CBUF_LEN-1] = '\0';
      chp = cbuf;
      do {
            if (*chp == '\n' || *chp == '\a' ) {
                  *chp = '\0';
            }
      } while (*chp++ != '\0');
      printf("\n\r");  fflush(stdout);
      if (!strncmp(cbuf, "exit", 4)) {
            terminate = 1;
      } else if (!strncmp(cbuf, "isr", 3)) {
            if (isr_run == 0) {
                // start scu timer
                printf(" * start irq61...\n\r");
                  R2_Copy |= MBINT_ENABLE;
                  MDHW_REG(2) = R2_Copy;
                isr_run = 1;
            } else {
                // stop scu timer
                printf(" * stop irq61...\n\r");
                  R2_Copy &= ~MBINT_ENABLE;
                  MDHW_REG(2) = R2_Copy;
                  isr_run = 0;
            }
            printf("ISR count: %d\n\r", ISR_Count);
      } else if (!strncmp(cbuf, "adc", 3)) {
            repeatc = 1;
            if ((isr_run !=0) && (strlen(cbuf) > 3)) {
```

```
            if (sscanf(&cbuf[3], "%d", &repeatc) != 1) {
                  repeatc = -1;
                  printf(" *** int conversion error.\n");
            }
      }
      for (k = 0; k < repeatc; k++) {
            printf("[%2d]: ", k);
            ADCRead();
            if (isr_run != 0) {
                  Do_Display = 0;
                  do {} while (Do_Display == 0);
            }
      }
} else if (cbuf[0]=='r') {
      repeatc = 1;
      if ((isr_run !=0) && (strlen(cbuf) > 1)) {
            if (sscanf(&cbuf[1], "%d", &repeatc) != 1) {
                  repeatc = -1;
                  printf(" *** int conversion error.\n");
            }
      }
      printf("  k     R0(OBP)   R1(DigIn)  R2(Status)     R3(ADC)\n");
      printf("-----------------------------------------------------\n");
      for (k = 0; k < repeatc; k++) {
            printf(" %2d", k);
                  xdata = MDHW_REG(0);  printf("    %8x", xdata);
                  xdata = MDHW_REG(1);  printf("    %8x", xdata);
                  xdata = MDHW_REG(2);  printf("    %8x", xdata);
                  xdata = MDHW_REG(3);  printf("    %8x\n", xdata);
            if (isr_run != 0) {
                  Do_Display = 0;
                  do {} while (Do_Display == 0);
            }
      }
      printf("-----------------------------------------------------\n");
} else if (cbuf[0]=='w') {
      if (sscanf(&cbuf[1], "%d %x", &regn, &xdata) != 2) {
            printf(" *** conversion error\n\r");
      } else {
            switch (regn) {
            case 0:
                  MDHW_REG(0) = xdata;
                  printf(" %8x -> R0 (OBP)\n\r", xdata);
                  break;
            case 1:
                  MDHW_REG(1) = xdata;
                  printf(" %8x -> R1 (DigOut)\n\r", xdata);
                  break;
            case 2:
                  MDHW_REG(2) = xdata;
                  printf(" %8x -> R2 (IntrCtrl)\n\r", xdata);
                  break;
            case 3:
                  MDHW_REG(3) = xdata;
                  printf(" %8x -> R3 (ADCDATA)\n\r", xdata);
                  break;
            default:
                  printf("*** illegal regnum\n\r");
            }
      }
}
```

```
      } while (terminate == 0);
    printf("shutting down...\n\r");

    MDHW_REG(2) = TIMER100ms;       // disable interrupt
      Xil_ExceptionDisable();
    XScuGic_Disable(&Intc, XPAR_FABRIC_MEASDIF_0_MD_IRQ_INTR);

      MDHW_REG(0) = 0;
    printf("Thank you for using MeasDem Tests V0.2a.\n\r");
    cleanup_platform();
    return 0;
}
```

---

Most of the source code is about interrupt setup. Linux applications are simpler in this case as interrupts are handled by the operating system kernel.

**Program example for Linux** (this is already included in  rootfs):

---

```
/**
 * @file   mdemus.c  MDemo Linux user space application
 * @author Kai Mueller, minor errors removed
 * @version 0.1
 * @brief  A Linux user space program that communicates with the
 * meascdd.c LKM. It passes a
 * string to the LKM and reads the response from the LKM.
 * For this example to work the device
 * must be called /dev/meascdd.
*/

#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<errno.h>
#include<fcntl.h>
#include<string.h>

#include "measdev.h"

// MEAScdd IO data
static MeasObj_struct TF_Obj_Snd, TF_Obj_Rcv;

#define CBUF_LEN 64
static char C_Buf[CBUF_LEN];

int main()
{
      int retc, fd;
      unsigned int cregn, cregval;
      int  terminate_me;

      printf("Starting meascdd test...\n");
      fd = open("/dev/meascdd", O_RDWR);  // Open the device with read/write
      if (fd < 0) {
            perror("Failed to open the device...");
            return errno;
      }
      // initialize send object
      TF_Obj_Snd.rnum = 0;
```

```
TF_Obj_Snd.rvalue = 0;

terminate_me = 0;
do {
	printf(">> "); fflush(stdout);
	fgets(C_Buf, CBUF_LEN, stdin);
	C_Buf[CBUF_LEN-1] = '\0';
	if (C_Buf[0] == 'x') {
		terminate_me = 1;
	} else if (C_Buf[0] == 'r') {
		if (sscanf(&C_Buf[1], "%d", &cregn) != 1) {
			printf("*** error converting int arg.\n");
		} else {
			TF_Obj_Snd.rnum = REGNUM_ID;
			TF_Obj_Snd.rvalue = cregn;
			printf(" o R%d set for reading.\n", cregn);
			// Write to LKM
			retc = write(fd, &TF_Obj_Snd, MEASOBJ_SIZE);
			if (retc < 0) {
				perror("Failed to write message to device.");
				close(fd);
				return errno;
			}
			// Read response from LKM
			retc = read(fd, &TF_Obj_Rcv, MEASOBJ_SIZE);
			if (retc < 0) {
				perror("Failed to read message from device.");
				close(fd);
				return errno;
			}
			printf("R%d: %x (hex)\n", TF_Obj_Rcv.rnum,
				 TF_Obj_Rcv.rvalue);
		}
	} else if (C_Buf[0] == 'w') {
		if (sscanf(&C_Buf[1], "%d %x", &cregn, &cregval) != 2) {
			printf("*** error converting ints.\n");
		} else {
			if (cregn >= NREGS) {
				printf("*** illegal register\n");
			} else {
				TF_Obj_Snd.rnum = cregn;
				TF_Obj_Snd.rvalue = cregval;
				printf(" o sending %x to REG[%d]\n",
					 cregval, cregn);
				retc = write(fd, &TF_Obj_Snd, MEASOBJ_SIZE);
				if (retc < 0) {
			perror("Failed to write the message to the device.");
					close(fd);
					return errno;
				}
			}
		}
	} else if (C_Buf[0] == 'i') {
		if (C_Buf[1] == '0') {
			printf(" => disable interrupt.\n");
			TF_Obj_Snd.rnum = 2;
			TF_Obj_Snd.rvalue = TIMER100ms;
			retc = write(fd, &TF_Obj_Snd, MEASOBJ_SIZE);
			if (retc < 0) {
		perror("Failed to write the message to the device.");
				close(fd);
				return errno;
```

```
                        }
                } else if (C_Buf[1] == '1') {
                        printf(" => enable interrupt.\n");
                        TF_Obj_Snd.rnum = 2;
                        TF_Obj_Snd.rvalue = MBINT_ENABLE | TIMER100ms;
                        retc = write(fd, &TF_Obj_Snd, MEASOBJ_SIZE);
                        if (retc < 0) {
                perror("Failed to write the message to the device.");
                                close(fd);
                                return errno;
                        }
                }
        } else {
                printf("*** unknown command\n");
        }
    } while (terminate_me == 0);

    printf(" => shutdown.\n");
    TF_Obj_Snd.rnum = 2;
    TF_Obj_Snd.rvalue = TIMER100ms;
    retc = write(fd, &TF_Obj_Snd, MEASOBJ_SIZE);
    if (retc < 0) {
        perror("Failed to write the message to the device.");
        close(fd);
        return errno;
    }
    close(fd);
    printf("Thank you for using mdemus.\n");
    return 0;
}
```

---

### 3.2.2 Petalinux

Petalinux is a pure command line tool for embedded Linux. It is based on the de-facto standard YOCTO including the BitBake-Layer (bb-layer) concept.

The main difference to YOCTO is the capability to import hardware information from Vivado and to generate boot files for tftp and SD-card.

Petalinux is used mainly in the beginning and the end of a project:

- create project folder
- Import hardware
- create and configure kernel
- create and configure rootfs
- create and program kernel modules (= device drivers)
- add applications to the rootfs after SW development
- build of kernel, modules, apps and the rootfs
- creating boot files

- build sdk (software development kit) for Vitis Linux application development
  (This ha sbeen done already)

All Petalinux tools must be done in the Petalinux project folder:

**~/ProjCanO/mcando/lxmcand**

---

Do not change anything in this folder and all subfolders. All modifications and custom applications as well as modules and device tree changes have to done in the recipes path:

**~/ProjCanO/mcando/lxmcand/project-spec/meta-user/**

---

If new boot files need to be generated the sequence is:

```
$ startpeta          (do this only once)
petalinux-package --boot --fsbl images/linux/zynq_fsbl.elf –fpga \
            images/linux/system.bit --u-boot –force
This creates:
      BOOT.BIN     (container file for FSB, bitfile, and u-boot)
      image.ub     (compressed linux image and device tree)
      boot.scr     (environment and boot devices for u-boot bootloader)
```

in **./images/linux/** .
These files have go on the SC-card: **./sdcardupdate** (script file):

---

```
#!/bin/bash

echo "Update bootfiles on SDCARD..."

BOOT_BIN_SOURCE=./images/linux/BOOT.BIN
IMAGE_UB_SOURCE=./images/linux/image.ub
BOOT_SCR_SOURCE=./images/linux/boot.scr
BOOT_BIN_TARGET=/media/dmplab/ZBOOT/BOOT.BIN
IMAGE_UB_TARGET=/media/dmplab/ZBOOT/image.ub
BOOT_SCR_TARGET=/media/dmplab/ZBOOT/boot.scr

if [ ! -f "$BOOT_BIN_SOURCE" ]; then
  echo "$BOOT_BIN_SOURCE does not exist."
  echo "Good-Bye."
  exit 1
fi
if [ ! -f "$IMAGE_UB_SOURCE" ]; then
  echo "$IMAGE_UB_SOURCE does not exist."
  echo "Good-Bye."
  exit 1
fi
if [ ! -f "$BOOT_SCR_SOURCE" ]; then
  echo "$BOOT_SCR_SOURCE does not exist."
  echo "Good-Bye."
  exit 1
fi
if [ ! -f "$BOOT_BIN_TARGET" ]; then
  echo "$BOOT_BIN does not exist."
  echo "Good-Bye."
  exit 1
fi
```

```
if [ ! -f "$IMAGE_UB_TARGET" ]; then
  echo "$IMAGE_UB does not exist."
  echo "Good-Bye."
  exit 1
fi
if [ ! -f "$BOOT_SCR_TARGET" ]; then
  echo "$BOOT_SCR_TARGET does not exist."
  echo "Good-Bye."
  exit 1
fi

rm $BOOT_SCR_TARGET
rm $IMAGE_UB_TARGET
rm $BOOT_BIN_TARGET

echo "Copying BOOT.BIN..."
cp $BOOT_BIN_SOURCE $BOOT_BIN_TARGET
echo "Copying image.ub..."
cp $IMAGE_UB_SOURCE $IMAGE_UB_TARGET
echo "Copying boot.scr..."
cp $BOOT_SCR_SOURCE $BOOT_SCR_TARGET

echo "update of bootfiles on SDCARD done."
```

---

The SD-card needs to be updated whenever the hardware has been changed.

---

---

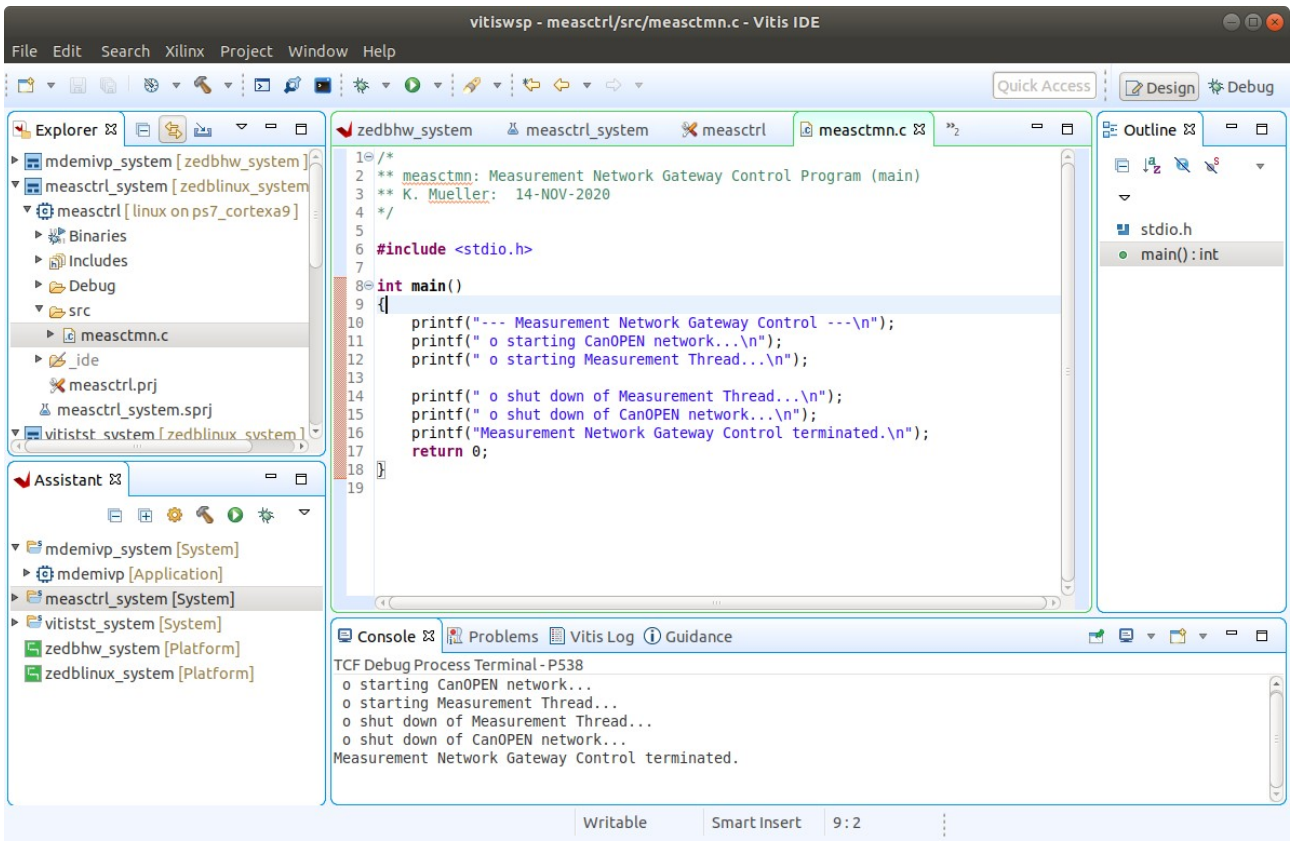
# 4 Development of Linux Programs for the ZedBoard Target System

Writing and executing programs on desktop Linux and on the target systems is very similar. The differences for the target system case are:

- An application program has to be created in *Vitis* for the *zedblinux_system* (all programs require an existing platform).
- The *zedblinux_system* platform provides cross compiler (i.e. *arm-xilinx-linux-gnueabi-clang* will be used instead of *gcc*) and the *binutils* tool-chain for target system development.
  The platform ensures that the search path for header files and libraries are mapped to the target system's installed libraries.
- Executing programs is done by TCF (target communication framework). This requires the setup of a TCF connection (only once – but needs to be modified if the IP address of the target changes due to DHCP).
  Therefore, the executable will be copied to the root file system. After a reboot all copied files are gone, as the target does not have (and don't need) a hard disk.

## 4.1 Creating a new Application

A new application program has to be assigned to the (existing) *zedblinux_system* platform. The following figure show the structure, the program and a successful program run on the target (all in the *Vitis* IDE).

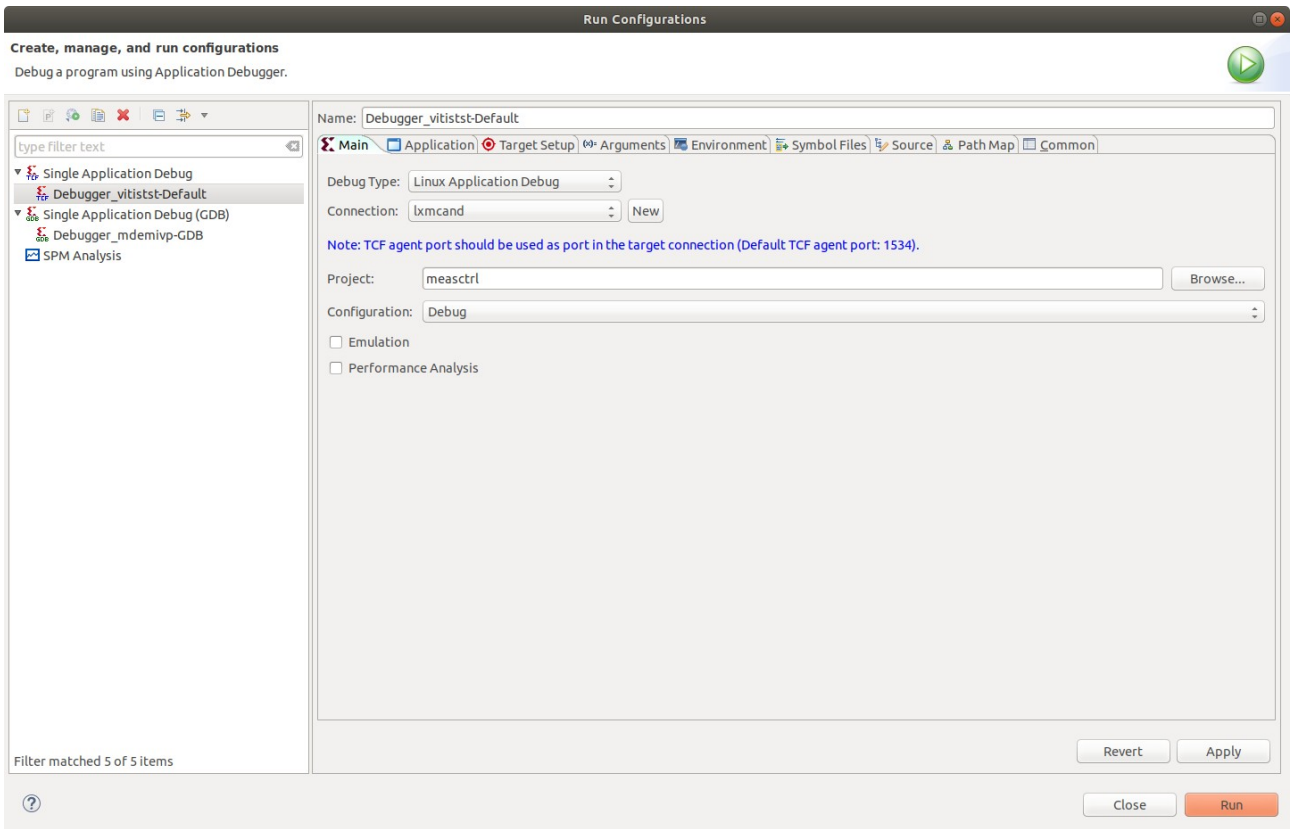



## 4.2 Configuring a TCF Connection

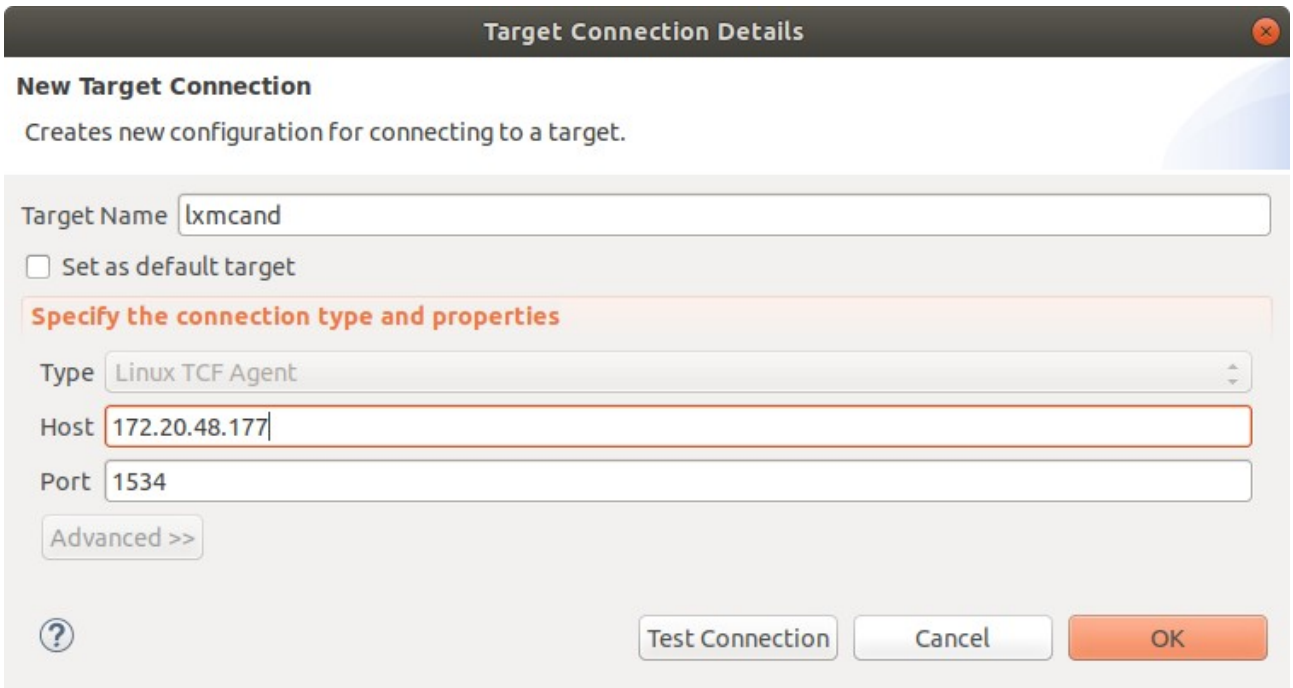
For program execution the target board must be up and running. On the target the TCF agent is started automatically. From the console terminal the assigned IP address (on *eth0*) can be seen:

```
/dev/ttyACM0 - PuTTY
root@lxmcand:~# ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defaul
t qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: can0: <NOARP,ECHO> mtu 16 qdisc noop state DOWN group default qlen 10
    link/can
3: can1: <NOARP,ECHO> mtu 16 qdisc noop state DOWN group default qlen 10
    link/can
4: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP gr
oup default qlen 1000
    link/ether 00:0a:35:00:1e:53 brd ff:ff:ff:ff:ff:ff
    inet 172.20.48.177/24 brd 172.20.48.255 scope global eth0
       valid_lft forever preferred_lft forever
    inet6 fe80::20a:35ff:fe00:1e53/64 scope link
       valid_lft forever preferred_lft forever
5: sit0@NONE: <NOARP> mtu 1480 qdisc noop state DOWN group default qlen 1000
    link/sit 0.0.0.0 brd 0.0.0.0
root@lxmcand:~#
root@lxmcand:~#
root@lxmcand:~#
```

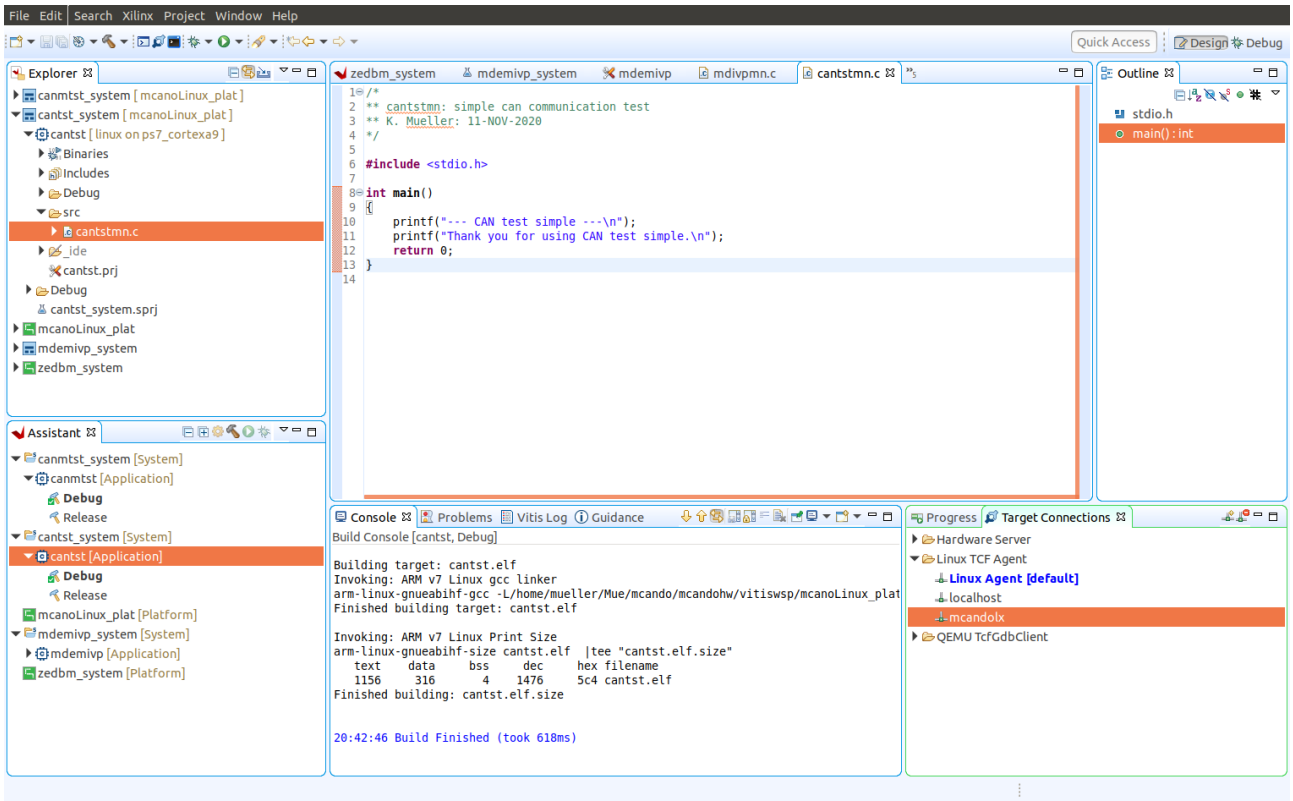
In the run configuration




the connection l*smcand* was configured as follows:




This connection definition needs to be changed only when the IP address of the target changes. In this case the connection requires changes from the connection list (bottom right of the following figure).




---

If the application program is final it should be included in the root file system (permanently).

---